



19.05.2025

DATA2410 Reliable Transport Protocol

Kandidatnummer: 112

Innhold

1. Introduksjon	3
2. Implementasjon av DRTP.....	3
2.1 Hva er DRTP og UDP	3
2.2 Klient funksjonen	3
2.3 Server funksjonen	5
2.4 Go-Back-N algoritme	5
3. Analyse av testresultatene: Sammenheng mellom vinduesstørrelse, RTT og throughput.....	6
3.1 Kjør filoverføringsprogrammet med vindusstørrelser 3, 5, 10, 15, 20 og 25.....	7
3.2 Analyse av pakketap ved bruk av -d.....	8
3.3 Hvor effektiv DRTP er med tc-netem	9
3.4 Explain what happens when a fin-ack packet is lost.....	11
Konklusjon:	12
Kilder:.....	13

1. Introduksjon

Denne rapporten dokumentere implementeringen av DRTP-DATA2410 Reliable transport Protocol. DRTP er en forenklet pålitelig transportprotokoll som er utviklet på toppen av UDP. Protokollen skal sørge for at data overføres på en sikker måte uten tap eller duplikater av pakker mellom klient og server.

Formålet med rapporten er å vise hvordan DRTP fungerer i praksis gjennom en filoverføringsapplikasjon. Programmet gjør det mulig å teste og analyse protokollens egenskaper i et kontrollert nettverksmiljø ved hjelp av Mininet. I prosjektet har jeg utviklet en løsning som håndter tilkoblingen, dataoverføring med Go-Back-N algoritme og til slutt avslutter forbindelsen.

Testingen har jeg utført med varierende vindusstørrelser, rundturstid (RTT) og pakketap. Hensikten med disse testene er å evaluere hvor sterk DRTP er under ulike nettverksituasjoner. Testene ble utført ved scriptet `simpe-topo.py` vi fikk som grunnlag for målinger av ytelse og feil behandling.

2. Implementasjon av DRTP

2.1 Hva er DRTP og UDP

For å utvikle DRTP har jeg laget et Python-program kalt `application.py` der både server og klient er samme fil. Hvilken rolle programmet kjører ved, styres av kommandolinjeflagg, for eksempel `-c` for klient eller `-s` for server. Kommunikasjonen skjer over UDP-sockets og pålitelighet er lagt til ved å implementere en egen protokoll kalt DRTP.

UDP User Datagram Protocol er en enkel og rask transportprotokoll som brukes til å sende data mellom enheter i et nettverk. I motsetning til TCP tilbyr ikke UDP noe garanti for at data kommer frem riktig rekkefølge. Så ved bruk av bare UDP er det ingen garanti for at pakken faktisk blir levert. Fordelen med UDP er at den er rask og gir lav forsinkelse. Derfor bygger DRTP på toppen av UDP for å håndtere pakkebekreftelser, riktig rekkefølger og eventuell tap av data.

```
create_header(seq_nr, ack_nr, flags, window):  
    return struct.pack(HEADER_FORMAT, seq_nr, ack_nr, flags, window)
```

Figur 1: bilde av `creat_header()`

Funksjonen `create_header()` lager en DRTP-header ved å pakke inn sekvensnr, bekræftelsenummer, flags og vindue størrelse. Dette gjøres med `struct.pack()` og en fast form `!HHHH`.

2.2 Klient funksjonen

Klient har ansvar for å sende en fil til mottakeren (serveren) ved hjelp av DRTP protokollen som er opprettet på toppen av UDP. Forbindelsen etableres gjennom en three-way handshake. Klienten sender

en SYN-pakke, serveren mottar SYN og svarer med SYN-ACK og klienten svarer tilbake med ACK. Når dette er fullført, er forbindelsen etablert.

Etter tilkoblingen blir filen delt opp i biter på 992 byte. Hver bit kombineres med en 8-byte DRTP-header som inneholder sekvensnummer, bekreftelsesnummer (ACK), flagg og vindusstørrelse. Klienten sender deretter pakkene til serveren. Klienten bruker sliding window mekanisme. I sliding window kan flere pakker bli sendt uten å vente på bekreftet (ACK). Vindusstørrelsen angis via kommandolinjeflagget -w. Hvis klienten ikke mottar en ACK innen 400 millisekunder, tolkes det som pakketap, og alle ubekreftede pakker i vinduet sendes på nytt. Klienten benytter Go-Back-N-prinsippet for å sende alt på nytt.

```
while next_seq < base + window_size and next_seq <= total_packets:
    start = (next_seq - 1) * DATA_SIZE
    end = start + DATA_SIZE
```

Figur 2: viser implementering av sliding window

Koden viser hvordan klienten implementer sliding window som gjøre det mulig å sende pakker så lenge vinduet ikke er full. Dette blir gjort for å gjøre det mulig å sende pakker uten å vente på bekreftelse som fører til bedre utnyttelse av nettverkskapasiteten.

Når alle pakkene er sendt og bekreftet, sender klienten en FIN-pakke for å avslutte forbindelsen. Dette skal signalisere at klienten ikke har mer data å sende. Det blir brukt en two-way handshake for å sikre at avslutningen skjer korrekt i DRTP.

Bilde under viser når klienten signalisere til sever om å avslutte forbindelsen:

```
#avslutter forbindelse ved å sende en FIN f
print("\nConnection Teardown:\n")
fin = create_header(0, 0, FLAGS['FIN'], 0)
sock.sendto(fin, server)
print("FIN packet is sent")
```

Figur 3: Klient sender en FIN pakke

2.3 Server funksjonen

Serveren har ansvarlig for å motta data over UDP og skrive innholdet til en fil. Den opprettes med en spesifikk IP-adresse og portnummer som oppgis av klienten via kommandolinjeargumenter. Serveren sender tilbake ACK-bekreftelser til klienten for å sikre at data mottas korrekt og i riktig rekkefølge.

Under vises et utdrag av koden der serverens UDP-socket opprettes og forbindelsen etableres:

```
def server(ip, port, discard_seq):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((ip, port))
    print(f"Server kjører på {ip}:{port}")

    # Connection Establishment
    data, client_addr = sock.recvfrom(PACKET_SIZE)
    seq_nr, ack_nr, flags, window = parse_header(data)
```

Figur 4: opprettelse av server og tilkobling

Her blir det opprettet en socket som bindes til IP adressen og portnr. Deretter venter serveren på en SYN-pakke fra klienten som starter den three-way handshake. Parse_header() brukes til å hente ut sekvensnr, flagg og vindusstørrelse fra pakken.

Serveren holder oversikt over forventet sekvensnummer og pakker som kommer i feil rekkefølge. Når overføringen er ferdig beregner den total overføringshastighet(throughput) som baseres på mottatt datamengde og varighet på forbindelsen.

2.4 Go-Back-N algoritme

DRTP bruker OG-Back-N algoritme for å sikre pålitelig og ordnet overføring av pakker over UDP. Denne algoritmen gjør det mulig å sende flere pakker uten å vente på bekreftelse for hver enkelt pakke ved å bruke sliding window. Hvis det skjer tap av en pakke eller ustabilitet innen 400ms blir alle pakkende sendt på nytt. Dette gjør Go-Back-N algoritmen velegnet til å håndtere pakketap i ustabile nettverk.

```
while next_seq < base + window_size and next_seq <= total_packets:
    start = (next_seq - 1) * DATA_SIZE
    end = start + DATA_SIZE
    chunk = file_data[start:end]
    packet = create_header(next_seq, 0, 0, 0) + chunk
    packets[next_seq] = packet
    sock.sendto(packet, server)
    print(f"{tidsstempel()} -- packet with seq = {next_seq} is sent, sliding window = {','.join(map(str, range(base, next_seq+1))}")
    next_seq += 1
```

Figur 5: Sending av pakker innenfor sliding window

Klienten sender pakker så lenge next_sq er innenfor vindusstørrelsen som er definert. Pakkene lagres i en ordbok packets i tilfelle de må sendes på nytt ved feil.

```

try:
    ack_data, _ = sock.recvfrom(PACKET_SIZE)
    _, ack_nr, flags, _ = parse_header(ack_data)
    if flags & FLAGS['ACK']:
        print(f"{tidsstempel()} -- ACK for packet = {ack_nr} is received")
        if ack_nr >= base:
            base = ack_nr + 1

```

Figur 6: Mottak av ACK og flytting av base

Når klienten mottar en bekreftelse (ACK) med gyldig sekvensnummer blir window flyttet frem ved å oppdatere base. Dette viser at pakken ble mottatt korrekt og at nye pakker kan sendes.

```

except socket.timeout:
    print("RTO occurred, retransmitting window...")
    for seq in range(base, next_seq):
        sock.sendto(packets[seq], server)
        print(f"{tidsstempel()} -- retransmitting packet with seq = {seq}")

```

Figur 7: Timeout og retransmisjon av vinduet

Hvis det ikke mottas noen ACK innen 400 ms, tolker klienten dette som pakketap (RTO). Da sender den alle pakker fra base til next_seq på nytt. Dette er kjernen i Go-Back-N algoritmen for å håndtere tap.

3. Analyse av testresultatene: Sammenheng mellom vinduesstørrelse, RTT og throughput

3.1 Kjør filoverføringsprogrammet med vindusstørrelser 3, 5, 10, 15, 20 og 25

Jeg har testa filoverføringsapplikasjonen i Mininet ved hjelp av simple-topo.py og har endret vindusstørrelsen med tre forskjellige RTT. RTT jeg har testa med er 50 ms, 100 ms og 200 ms. I alle tilfellene ble den samme bildefilen overført fra klienten til server. Resultatene for throughput målt i Mbps vises i tabellen under.

RTT	-w 3	-w 5	-w 10	-w 15	-w 20	-w 25
100	0.22 Mbps	0.39 Mbps	0.74 Mbps	1.05 Mbps	1.09 Mbps	1.05 Mbps
200	0.12 Mbps	0.19 Mbps	0.39 Mbps	0.58 Mbps	0.58 Mbps	0.58 Mbps
50	0.46 Mbps	0.75 Mbps	1.46 Mbps	2.20 Mbps	2.15 Mbps	2.17 Mbps

Tolkning av Resultatene:

Resultatene viser tydelig at større vindusstørrelse fører til økt throughput spesielt når RTT er lav. Dette er fordi vindusstørrelse gir muligheten for at flere pakker kan sendes inn før man på vente på bekreftelse (ACK). Dette er samme prinsipp som TCP bruker i sin sliding window-mekanisme (Islam, 2025) og er også kjernen i Go-Back-N algoritmen som DRTP bygger på.

Når RTT er 100 ms øker throughput jevnt helt til vindusstørrelsen er 15. Etter dette flater ytelsen ut og målingene for -w 20 og -w 25 er nesten like. Dette er fordi serveren mottakervindu er satt til 15 og klienten automatisk tilpasse seg denne grensen. Derfor har ikke høyere vindusstørrelse noe praktisk effekt.

Når vi øker RTT verdien 200 ms ser vi throuputen verdiene synker. Dette stemmer med teorien om retransmisjonstid(RTO)(Islam, 2025), som sier at høy RTT fører til lengere venting på ACK. Dette reduserer effektiviteten spesielt når vindusstørrelsen er liten. Dette ser vi fra tabellen når vindusstørrelsen er 3 er throughput 0.12Mbps, vi ser også selv med høy vindusstørrelse er throughput ikke noe større.

De beste resultatene oppnås når RTT er 50 ms. Høyest målte throughput er 2.17 Mbps ved -w 25. Dette viser at lav RTT kombinert med stor vindusstørrelse gir maksimal utnyttelse av nettverket. Likevel kan vi også se det her at verdier høyere enn -w 15, holde throughput seg lik.

```
syn_ack = create_header(0, 0, FLAGS['SYN'] | FLAGS['ACK'], 15)
sock.sendto(syn_ack, client_addr)
print("SYN-ACK packet is sent")
```

Figur 8: Server sender SYN-ACK med vindusstørrelse = 15

Når forbindelsen etableres, sender serveren en SYN-ACK pakke med vindusstørrelsen er satt til 15 som vises i figuren 8. Denne verdien informerer klienten hvor mange pakker klienten kan sende uten å vente

på ACK. I praksis betyr det at selv om klienten setter høyere vindusstørrelsen må den tilpasse seg serverens begrensning. Dette forklarer hvorfor resultatene til throughputen ikke øker etter -w 15.

Disse målingene bekrefter at justering av vindusstørrelse og RTT har mye å si for ytelsen til DRTP. Når RTT er høy krever det større vindu for å kompensere for forsinkelse, men lav RTT gir en vindusstørrelse bedre ytelse.

3.2 Analyse av pakketap ved bruk av -d

For å demonstrere hvordan DRTP håndterer pakketap på pålitelig måte, brukte jeg -d 1830 i serveren.

Det gjør at pakken med sekvensnummer 1830 ble ignorert. Figur 9 viser at 1830 pakken ikke ble akseptert og at etterfølgende pakker også ble forkastet. Etter en stund oppstår det RTO (Retransmission Timeout) og klienten starter på nytt ved å sende alle pakker fra og med 1830 på nytt.

Dette følger Go-Back-N algoritmen jeg forklarte over, der tap av en pakke gjør at følgende pakker i vinduet må sendes på nytt. Når klienten sender pakker på nytt, aksepterer serveren dem og svarer med en ACK, som vist i Figur 10. Overføringen fortsetter deretter normalt.

```
21:38:42.182887 -- sending ack for the received 1829
Discarding packet 1830 for testing
Out of order packet 1831 is received -- ignored
Out of order packet 1832 is received -- ignored
Out of order packet 1833 is received -- ignored
Out of order packet 1834 is received -- ignored
21:38:42.284198 -- packet 1830 is received
21:38:42.284964 -- sending ack for the received 1830
21:38:42.287118 -- packet 1831 is received
21:38:42.288002 -- sending ack for the received 1831
21:38:42.289547 -- packet 1832 is received
21:38:42.290223 -- sending ack for the received 1832
21:38:42.296779 -- packet 1833 is received
21:38:42.297805 -- sending ack for the received 1833
21:38:42.299539 -- packet 1834 is received
21:38:42.300342 -- sending ack for the received 1834
21:38:42.388172 -- packet 1835 is received
21:38:42.390448 -- sending ack for the received 1835
21:38:42.391835 -- packet 1836 is received
21:38:42.393147 -- sending ack for the received 1836
21:38:42.400017 -- packet 1837 is received
21:38:42.401081 -- sending ack for the received 1837
21:38:42.405033 -- packet 1838 is received
21:38:42.405507 -- sending ack for the received 1838
21:38:42.405633 -- packet 1839 is received
21:38:42.406688 -- sending ack for the received 1839
21:38:42.492215 -- packet 1840 is received
21:38:42.493140 -- sending ack for the received 1840
21:38:42.503311 -- packet 1841 is received
21:38:42.519553 -- sending ack for the received 1841
FIN packet is received
FIN-ACK packet is sent
The throughput is 0.36 Mbps
Connection closed
root@ubuntu:/home/ubuntu/Data2410Eksamen-main#
```

Figur 9: viser klientens h1 som ignorerer pakke 1830.


```

1832, 1833, 1834}
RTO occurred, retransmitting window...
21:38:42.183650 -- retransmitting packet with seq = 1830
21:38:42.183742 -- retransmitting packet with seq = 1831
21:38:42.185516 -- retransmitting packet with seq = 1832
21:38:42.185579 -- retransmitting packet with seq = 1833
21:38:42.186164 -- retransmitting packet with seq = 1834
21:38:42.285075 -- ACK for packet = 1830 is received
21:38:42.287273 -- packet with seq = 1835 is sent, sliding window = {1831, 1832,
1833, 1834, 1835}
21:38:42.288525 -- ACK for packet = 1831 is received
21:38:42.290008 -- packet with seq = 1836 is sent, sliding window = {1832, 1833,
1834, 1835, 1836}

```

Figur 10: viser serveren h2

3.3 Hvor effektiv DRTP er med tc-netem

For å undersøke hvor effektiv DRTP er under reelle nettverksforhold med pakketap. Brukte tc-netem i Mininet for å simulere pakketap. Jeg endret simple-topo.py ved å kommentere ut linje RTT delay 100ms og erstattet den med delay 100ms loss X%. Der X var henholdsvis 2%, 5% og 50%. Under testingen ble samme bilde fil sendt fra klient til server med ulike vindusstørrelser.

Resultatet til 2% loss rate:

RTT	-w 3	-w 5	-w 10	-w 15	-w 20	-w 25
100	0.19 Mbps	0.29 Mbps	0.47 Mbps	0.47	0.48	0.50

Når pakketap er 2% ser vi at DRTP er relativt god ytelse. Throughput øker som forventet i takt med økt vindusstørrelse, fra 0.19 Mbps ved -w 3 til 0.47 Mbps ved -w 15. Vi ser at økningen stopper opp etter -w 15 noe som stemmer med tidligere funn 3.1 som var at serverens vindusstørrelse er begrenset til 15. Resultatene viser at protokollen klarer å håndtere moderate pakketap effektivt ved hjelp av Go-Back-N-algoritmen som sørger for at tapte pakker blir oppdaget og sendt på nytt.

Resultatet til 5% loss rate:

RTT	-w 3	-w 5	-w 10	-w 15	-w 20	-w 25
100	0.14	0.19	0.24	0.28	0.29	0.23

Ved 5% pakketap ser vi at det har skjedd reduksjon i throughput spesielt for lave vindusstørrelser. For eksempel gir -w 3 kun 0.14 Mbps. Selv ved høyere verdier som -w 20 og -w 25 er throughput 0.29 og 0.23 Mbps som er lavere enn resultatet for 2% pakketap. Dette er på grunn at flere pakker går tapt som fører til effektiv RTO og lavere utnyttelse av båndbredden. DRTP klarer fortsatt å overføre hele filen, men effektiviteten er redusert.

Resultatet til 50% loss rate:

RTT	-w 3	-w 5	-w 10	-w 15	-w 20	-w 25
100	0	0	0	0	0	0.02 Mbps

Ved 50 % pakketap klarte DRTP kun å gjennomføre filoverføringen når vindusstørrelsen var satt til -w 25. I alle andre tilfeller mislyktes overføringen fullstendig, og klienten havnet i en kontinuerlig retransmisjonssyklus. Selv ved -w 25 ble throughputen svært lav, kun 0.02 Mbps, og det tok svært lang tid å fullføre overføringen.

Dette viser svakheten Go-Back-N-algoritmen når pakketap er høyt. Siden hele vinduet må starte på nytt når en pakke går tapt, fører dette til mye unødvendig trafikk og forsinkelser. Samtidig viste dette at en større vindusstørrelse gir større sjanse for å overleve massivt pakketap, dette ser vi i tabellen når vindusstørrelsen 25 er den eneste som fikk til å overføre data. Fordi flere pakker kan være i omløp, og sannsynligheten for fremgang øker marginalt.

DRTP benytter et treveis håndtrykk (three-way handshake) for å etablere forbindelser og et toveis håndtrykk (two-way handshake) for å avslutte dem. Kontrollpakken som SYN, ACK, SYN-ACK og FIN er avgjørende for korrekt opprette og avslutte forbindelse. Men hvis en av disse pakken går tapt oppstår det problem.

Figurene 11 og 12 viser eksempel på ACK eller SYN-ACK går tapt under opprettelsen av forbindelsen. Det vi ser i figuren er at SYN-ACK ble sendt fra server, men ikke mottatt av klienten. Resultatet er at klienten gir opp og viser meldingen *"Timeout during handshake"*, mens serveren avslutter forsøket. Dette viser at DRTP er svak hvis pakketap er høy.

```

SYN packet is sent
Timeout during handshake
root@ubuntu:/home/ubuntu/Data2410Eksamen-main#

```

Figur 11: klient h1 ser at SYN packet er sent

```

SYN packet is received
SYN-ACK packet is sent
Expected ACK, connection failed.
root@ubuntu:/home/ubuntu/Data2410Eksamen-main#

```

Figur 12: server h2 SYN er mottatt og SYN-ACK er sendt

under viser en vellykket two-way handshake og three-way handshake: Både server og klienten mottar riktig pakke

```
FIN packet is sent
FIN-ACK packet is received
The throughput is 0.25 Mbps
Connection closed
```

Figur 13: klient h1

```
10.10.10.10:50250 -> sending ack for the received 1042
FIN packet is received
FIN-ACK packet is sent
The throughput is 0.25 Mbps
Connection closed
root@ubuntu:/home/ubuntu/Data2410Eksamen-main#
```

Figur 14: server h2

3.4 Explain what happens when a fin-ack packet is lost.

DRTP benytter en toveis håndtrykksmekanisme (two-way handshake) for å avslutte forbindelser.

Klienten signaliserer at den er ferdig med overføringen ved å sende en FIN-pakke, og serveren svarer med en FIN-ACK-pakke som både bekrefter mottaket og initierer sin egen avkobling.

Hvis FIN-ACK pakken går tapt underveis i nettverket vil ikke klienten motta bekreftelse på at serveren av avsluttet forbindelsen. Som fører til at klienten blir satt i en ventetilstand og lukker ikke forbindelsen på en riktig måte. Hvis klienten ikke mottar pakken innen en gitt tidsgrense vil den automatisk avslutter forbindelsen gjennom timeout. Dette ser vi godt i eksempel om pakketap på 50% , den kunne ikke håndtere pakketapet og FIN-ACK ble ikke mottatt.

Ifølge prinsippene for tilstandsoverganger i transportlaget (Islam, 2025) er det viktig både pakker og kontrollspakene blir behandlet riktig. Fra testene jeg har gjort ser vi at DRTP ikke er effektiv for løsning av høy pakketap. Den kan bare håndtere lavt pakketap. En forbedring til neste gang kan være å legge til at FIN-ACK sendes hele tiden til bekreftelse er mottatt.

Konklusjon:

I denne rapporten har jeg utviklet og testet DRTP en forenklet, pålitelig transprotokoll bygget på toppen av UDP. Jeg har gjennomført ulike test scenarier gjennom Mininet for å evaluere hvor effektiv DRTP er under ulike nettverksituasjoner, inkludert ulike RTT, pakketap og varierende vindusstørrelser. Gjennom disse testene fikk jeg innsikt i hvordan protokollen fungerer i praksis.

I den første testserien observerte jeg at økt vindusstørrelse fører til høyere throughput, spesielt når RTT er lav. Det jeg observerte var at throughput flatet seg etter vindusstørrelsen 15 og dette var på grunn av at serveren har et mottakervindu på 15. Dette viser hvor viktig det er at sender og mottaker kommunisere sine vindusstørrelser.

Ved bruk av tc-netem simulerte jeg pakke tap og analysere hvordan DRTP håndterte dette. Ut ifra resultatene ser vi ved 2% og 5% pakketap, kunne DRTP protokollen håndtere dette ved Go-Back-N algoritmen som sender vindu på nytt. Men ved 50% pakke tap kunne ikke protokollen håndtere den. Dataoverføringen fungerer bare med vindusstørrelsen 25 og det var med svært lav ytelse på 0.02 Mbps. Dette viser Go-Back-N algoritmen svakheten i miljø med høyt pakke tap. Testet også -d flagges som ignorerer en spesifikk pakke. Det som skjedde, var at klienten retransmitterte hele vinduet fra den tapte pakken.

Ut ifra testene ser vi at DRTP protokollen fungerer som en enkel og effektiv løsning for filoverføring. Protokollen ga en praktisk innsikt i hvordan man lager tilkoblingen, dataoverføring med Go-Back-N algoritme og til slutt avslutter forbindelsen riktig.

Kilder:

Islam, S. (2025, februar 28). *Transmission Control Protocol*. file:///C:/Users/zubey/Downloads/transport-layer-tcp.pdf

<https://github.com/safiqu/2410/blob/main/docs/netem/mininet-tc-delay.md>